

Contents

Yelp Restaurants — Module 3 Lab Guide	2
The Scenario — Your Mission	2
Where you work	2
The problem	2
Your manager's request	3
Your team's job for the next 2 weeks (Module 3)	3
After Module 3	3
A note about size	3
How This Guide Works (Read This First!)	4
We do NOT give you the full code	4
Visualizations — Two Modes	4
1. Exploratory charts (for YOU)	4
2. Explanatory charts (for PRIYA)	4
Your plotting toolkit (you learned this in Module 2 Class 5)	4
Before You Start — Google Colab Setup	5
Step 1 — Open a new Colab notebook	5
Step 2 — Connect to Google Drive	5
Step 3 — Create a project folder	5
Step 4 — Get the data	5
Step 5 — Test it (read JSON, not CSV!)	6
Step 6 — Tell Colab to give you more memory	6
Colab tips	6
Class 1 — Data Cleaning	6
Your goal	6
Inputs	6
Outputs	7
Phase A — Explore the data first (15 minutes)	7
Phase B — Filter to restaurants in one city (45 minutes)	7
Phase C — Make ONE chart for Priya (15 minutes)	9
Common mistakes in Class 1	9
Self-check before Class 2	9
Class 2 — Encoding and Scaling	10
Your goal	10
Inputs	10
Outputs	10
Phase A — Explore the data first	10
Phase B — Encode and scale	11
Phase C — Make ONE chart for Priya	13
Common mistakes in Class 2	13
Self-check before Class 3	13
Class 3 — Feature Engineering	13
Your goal	13
Inputs	14
Outputs	14
Phase A — Explore the data first	14
Phase B — Engineer the features	14
Phase C — Make ONE chart for Priya	17
Common mistakes in Class 3	17
Self-check before Class 4	18

Class 4 — Feature Selection	18
Your goal	18
Inputs	18
Outputs	18
Phase A — Explore the data first	18
Phase B — Select features	19
Phase C — Make ONE chart for Priya	20
Common mistakes in Class 4	20
Self-check before Class 5	21
Class 5 — Pipelines	21
Your goal	21
Inputs	21
Outputs	21
Phase A — Explore the data first	21
Phase B — Build the pipeline	22
Phase C — Make ONE chart for Priya	23
Common mistakes in Class 5	23
Self-check before Class 6	23
Class 6 — End-to-End Lab (THE BIG ONE)	24
Your goal	24
Time	24
Inputs	24
Outputs	24
Phase A — Explore the data first (10 minutes)	24
Phase B — Build the final dataset (70 minutes)	24
Phase C — Findings report for Priya (10 minutes)	25
Grading rubric (100 points)	26
Submit	26
Tips for the whole module	26

Yelp Restaurants — Module 3 Lab Guide

Scenario: Yelp Open Dataset. Predict the star rating of a restaurant. **Module:** 3 (Data Preparation and Feature Engineering) **Audience:** Adult learners, A2 English. New to AI. **Tools:** Google Colab (no install needed).

The Scenario — Your Mission

Where you work

You and your team are new data analysts at **Yelp**. Yelp is a big website in the USA. People go to Yelp to find a restaurant, a cafe, a bar, or a shop. They read reviews. They look at the star rating (from 1 to 5 stars).

Every month: - **Millions of people** search on Yelp. - **150,000 businesses** are listed. - **About 50,000** of them are restaurants. - People write **6 million+ reviews**.

The star rating (1.0 to 5.0, in steps of 0.5) is the most important number. A restaurant with 4.5 stars gets many customers. A restaurant with 2.0 stars gets very few.

The problem

When a **new restaurant** opens, Yelp has a problem: - The restaurant has **zero reviews**. - So the rating is “**unknown**”. - The website shows it at the bottom of search results, or hides it. - Customers do not see it. They do not visit. The

restaurant does not get reviews. - **30% of new restaurants close in the first 90 days.**

This is bad for Yelp: - The restaurant pays Yelp for ads, but gets nothing. - The restaurant owner tells friends: “Yelp is useless for new places.” - Yelp loses money and reputation.

Your manager’s request

Your manager, **Priya** (VP of Local Search Quality at Yelp HQ), tells you:

“When a new restaurant opens, our search ranking has no rating yet. We give it ‘unknown’ stars and hide it from results. We lose 30% of new restaurants in the first 90 days.

Build a model that predicts the star rating **from category, price, location, and metadata** — before the first review arrives.

If the model predicts 4.2 stars, we put the restaurant at the top. If it predicts 2.1 stars, we put it lower. We can change this later, when real reviews come in.

I do not need 100% perfect. I need a good guess. Anything is better than ‘unknown’.”

Your team’s job for the next 2 weeks (Module 3)

Priya cannot do this alone. Her data is in **5 big JSON files**. JSON is a format that looks like text. It is harder to read than CSV. Some columns are **nested** (a column inside a column). Some columns are **missing 90% of the time**.

Your job in Module 3: > **Turn 5 messy JSON files into ONE clean file. The clean file will be used to train the model in Module 4.**

The clean file is called `yelp_restaurants_clean.parquet`. It must have **22 specific columns** (we will see them in Class 6).

After Module 3

Module	What happens
Module 4	Train the prediction model. Priya finally gets her “star score” for new restaurants.
Module 5	Find groups of restaurants (cheap-and-good vs expensive-and-bad, etc.). For marketing.
Module 7	Read the review text (in English). Find common complaints (slow service, cold food, etc.).

You use the **same Yelp dataset** until the end of Module 7.

A note about size

The full Yelp dataset is **6 million reviews and 150,000 businesses**. This is too big for Google Colab free tier. So in Module 3 we will: - **Filter to restaurants only** (about 50,000 rows). - **Filter to ONE city** (Philadelphia, Tampa, or another big city in the data). This gives us about **10,000 restaurants** and a few hundred thousand reviews. Small enough for Colab.

We will write code that **works for any city**. So in Module 6, we can run the same code on the full dataset.

How This Guide Works (Read This First!)

We do NOT give you the full code

In this guide you will see two kinds of content:

1. WHAT / WHY / EXPECTED — explained in words

For every step you read: - **WHAT** you have to do (in normal language) - **WHY** you do it (the reason) - **HINTS** — the function names, the arguments to remember - **EXPECTED** — the result you should see if you did it right

2. Sometimes a tiny code skeleton with blanks

For the harder steps we give you a code skeleton with ____ blanks. **You fill in the blanks.**

Why no full code?

Because you will NOT learn if you copy-paste. - In Module 4, the model only works if YOU understand each step. - In your future job, nobody will give you the code. - Writing the code yourself takes 10 minutes more today, but saves 10 hours later.

Rule: If you finish a step in 30 seconds by copy-pasting from a teammate's notebook, do it AGAIN by yourself. Different variable names, same logic.

Visualizations — Two Modes

In every class you will make charts. There are **two reasons** to make a chart:

1. Exploratory charts (for YOU)

Made BEFORE you clean the data. To understand what the data looks like. - Fast, ugly, no labels needed. - Examples: `df.hist()`, `df['column'].value_counts().plot.bar()`.

Goal: “**What does this data look like?**”

2. Explanatory charts (for PRIYA)

Made AFTER you understand. To EXPLAIN something to your manager. - Clean, labeled, one clear message. - Must have: title, x-label, y-label, source, and a 1-sentence takeaway.

Goal: “**Priya, look at this. This is what the data says.**”

In every class you make BOTH kinds.

Your plotting toolkit (you learned this in Module 2 Class 5)

Plot	When to use it	Function name
Histogram	See the SHAPE of a numeric column	<code>plt.hist()</code> or <code>sns.histplot()</code>
Bar chart	Count of each category	<code>df['col'].value_counts().plot.bar()</code>
Box plot	See outliers in a numeric column	<code>sns.boxplot()</code>
Scatter plot	See the relationship between 2 numbers	<code>plt.scatter()</code> or <code>sns.scatterplot()</code>
Heatmap	See correlation between many columns	<code>sns.heatmap(df.corr())</code>

Plot	When to use it	Function name
Map / geo scatter	Show location on a map	<code>plt.scatter(lon, lat)</code>

Before you start: `import matplotlib.pyplot as plt` and `import seaborn as sns`.

Before You Start — Google Colab Setup

You will work in **Google Colab**. That means: - You do NOT install Python. - You do NOT install pandas, scikit-learn, or anything. - You just open a notebook in your web browser.

Step 1 — Open a new Colab notebook

1. Go to <https://colab.research.google.com>
2. Sign in with your Google account.
3. Click “**New notebook**” (top left).
4. Name it `yelp_module_3.ipynb`.

Step 2 — Connect to Google Drive

Colab files **disappear** when you close the browser. So you must save your work to **Google Drive**.

In the first cell:

```
from google.colab import drive
drive.mount('/content/drive')
```

A pop-up asks for permission. Click “Allow”. After this, Drive is available at `/content/drive/MyDrive/`.

Step 3 — Create a project folder

In a new cell:

```
import os
os.makedirs('/content/drive/MyDrive/yelp_lab', exist_ok=True)
%cd /content/drive/MyDrive/yelp_lab
```

Your team will save ALL files in this folder.

Step 4 — Get the data

The Yelp Open Dataset is at <https://www.yelp.com/dataset>

It is **about 4 GB** when unzipped. This is big. Be patient.

Option A — Download from the Yelp website (recommended):

1. Go to <https://www.yelp.com/dataset>.
2. Click “Download Dataset”. Sign the agreement (academic + non-commercial use only).
3. You will get a file `yelp_dataset.tar` (about 4 GB).
4. Upload it to your Google Drive (not to Colab — Drive is permanent).
5. In Colab:

```
%cd /content/drive/MyDrive/yelp_lab
!tar -xf yelp_dataset.tar
!ls
```

You should see **5 JSON files**: - yelp_academic_dataset_business.json - yelp_academic_dataset_review.json - yelp_academic_dataset_user.json - yelp_academic_dataset_checkin.json - yelp_academic_dataset_tip.json

Option B — Use a smaller mirror (if download is slow):

Some teachers prepare a smaller version (only one city, already filtered). Ask your mentor.

Step 5 — Test it (read JSON, not CSV!)

JSON files are different from CSV. Each line of the file is one JSON object (one row). This is called **JSON Lines** or **NDJSON**.

In Colab:

```
import pandas as pd
business = pd.read_json('yelp_academic_dataset_business.json', lines=True)
print(business.shape)
```

Should print something like (150346, 14). **150,000 businesses, 14 columns**. You are ready.

Important: lines=True tells pandas “read line by line”. Without it, pandas tries to read the whole file as one big JSON object and crashes.

Step 6 — Tell Colab to give you more memory

The full dataset is big. Click **Runtime → Change runtime type → High-RAM** if available (free in Colab for now).

Colab tips

Tip	Why
Save your notebook to Drive often	Colab disconnects after 12 hours.
Save intermediate .parquet files to Drive	Files in /content/ disappear when Colab disconnects.
Share the notebook with teammates (Share button, top right)	Editor access. Like Google Docs for code.
Restart runtime if memory full	Runtime → Restart runtime.
Filter to ONE city as early as possible	Saves memory. The full review file is 5 GB.

Class 1 — Data Cleaning

Scenario reminder: Priya drops 5 big JSON files on your desk. The files include all kinds of businesses (dentists, salons, gyms, restaurants). Your job today: keep only restaurants, in one city, and understand what columns we have.

Your goal

Make the 5 JSON files **USABLE**. Filter to restaurants. Filter to ONE city. Look at what columns exist.

Inputs

- The 5 JSON files in /content/drive/MyDrive/yelp_lab/

Outputs

- `business_step1.parquet` saved in your Drive folder
 - 3+ exploratory charts in your notebook
 - 1 explanatory chart for Priya
 - Notes in markdown about every decision you made
-

Phase A — Explore the data first (15 minutes)

Before you clean anything, **LOOK** at the data.

Exploratory chart 1 — How many businesses per city?

- **Question:** “Yelp has businesses in many cities. Which cities have the most?”
- **HINTS:**
 - Use `business['city'].value_counts().head(20)`.
 - Then `.plot.bar()` on the result.
- **What you learn:** The top 5 cities have most of the data. We will pick ONE city.

Exploratory chart 2 — How many businesses per state?

- **Question:** “Which US state has the most Yelp businesses?”
- **HINTS:**
 - Use `business['state'].value_counts()`.
 - Plot as a bar chart.
- **What you learn:** Yelp covers many states. Most data is from a few big ones.

Exploratory chart 3 — The star rating distribution

- **Question:** “What is the shape of the star column? Is it normal? Skewed?”
- **HINTS:**
 - Use `business['stars'].value_counts().sort_index().plot.bar()`.
 - The stars are 1.0, 1.5, 2.0, ... up to 5.0 (in 0.5 steps).
- **What you learn:** Most restaurants have 3.5 to 4.5 stars. Very few have 1.0 or 5.0. This is the target column for Module 4.

Exploratory chart 4 — Missing values per column

- **Question:** “Which columns are mostly missing?”
 - **HINTS:**
 - Use `business.isna().sum().sort_values(ascending=False)`.
 - Plot it as a bar chart.
 - **What you learn:** Some columns (like hours) are missing for many rows. We must decide what to do.
-

Phase B — Filter to restaurants in one city (45 minutes)

Step 1 — Load the business file

- **WHAT:** Load `yelp_academic_dataset_business.json` into a DataFrame called `business`.
- **HINTS:**
 - Use `pd.read_json(..., lines=True)`.
 - Remember the `lines=True` argument. Without it the code crashes.
- **EXPECTED:** Around 150,346 rows and 14 columns.

Step 2 — Look at the columns

- **WHAT:** Check `.shape`, `.info()`, and `.head()` of `business`.
- **HINTS:**
 - Pay attention to the column types (Dtype column in `.info()`).
 - Are attributes and hours stored as object? They are nested dictionaries inside cells. We will see this later.
- **EXPECTED:** Columns include: `business_id`, `name`, `address`, `city`, `state`, `latitude`, `longitude`, `stars`, `review_count`, `is_open`, `attributes`, `categories`, `hours`.

Step 3 — Filter to RESTAURANTS only

This is the most important filter. The `categories` column is a TEXT string like:

"Mexican, Bars, Restaurants, Nightlife"

We want to keep any row where the text contains the word "Restaurants".

- **WHAT:** Keep rows where `categories` contains the word "Restaurants".
- **HINTS:**
 - First, some rows have `None` (missing) in `categories`. You must handle this.
 - Use `business['categories'].fillna('')` to replace `None` with empty string.
 - Then use `.str.contains('Restaurants', case=False, na=False)`.
 - Boolean mask, then filter.
- **WHY:** Yelp has dentists, hair salons, gyms, etc. We only care about restaurants.
- **EXPECTED:** About **52,000 rows** left.

Step 4 — Look at top cities AFTER filtering

- **WHAT:** Now we have only restaurants. Where are they?
- **HINTS:** `restaurants['city'].value_counts().head(10)`.
- **EXPECTED:** Cities like Philadelphia, Tampa, Tucson, Indianapolis, Nashville.

Step 5 — Pick ONE city

Pick the city with the most restaurants. (Or follow your mentor's choice. Usually Philadelphia or Tampa.)

- **WHAT:** Keep only rows where `city == 'Philadelphia'` (or your chosen city).
- **HINTS:**
 - Simple boolean mask: `restaurants[restaurants['city'] == 'Philadelphia']`.
 - Add `.copy()` at the end. This avoids a warning later.
 - Save the city name in a variable: `CITY = 'Philadelphia'`. Use it everywhere.
- **WHY:** The full restaurant table is 50,000 rows. With reviews, this is too big for Colab. One city gives us ~5,000 to ~10,000 restaurants. Small enough.
- **EXPECTED:** About **5,000 to 10,000 rows** left, depending on the city.

Step 6 — Filter to OPEN restaurants only (optional but recommended)

- **WHAT:** The column `is_open` is 1 if the restaurant is still open, 0 if closed.
- **HINTS:** `restaurants[restaurants['is_open'] == 1]`.
- **WHY:** Closed restaurants are not useful for Priya. She wants to predict ratings of NEW open restaurants.
- **NOTE:** You can also keep closed ones for analysis. Discuss with your team. Write down your choice.

Step 7 — Find missing values

- **WHAT:** Count missing values per column.
- **HINTS:** `.isna().sum()` per column. Sort it.
- **EXPECTED:** `attributes`, `hours`, `address` may have some missing. `stars`, `business_id`, `latitude`, `longitude` should be 0 missing.

Step 8 — Write down what you did

In a markdown cell, write: - Starting rows: ~150,346 businesses - After “Restaurants” filter: ~52,000 - After city filter: ~6,000 (your number) - After `is_open` filter: ~5,000 (your number) - WHY you removed each group.

Step 9 — Save to Drive

- **WHAT:** Save your filtered business DataFrame as parquet.
- **HINTS:** `df.to_parquet('/content/drive/MyDrive/yelp_lab/business_step1.parquet')`.
- **WHY parquet, not CSV?** Parquet is smaller and faster. It keeps column types (CSV loses them).

Phase C — Make ONE chart for Priya (15 minutes)

She does not have time to read your code. She wants ONE picture.

Priya’s chart — “How did we filter the data?”

Make a bar chart showing the row count after each filter step:

Step	Rows
All businesses	150,346
After “Restaurants” filter	~52,000
After city filter	~6,000
After “is_open” filter	~5,000

- **HINTS:**
 - Use `plt.bar()` with 4 bars.
 - Add the title: “Yelp data filtering – we kept the right slice”.
 - X-label: filter step name.
 - Y-label: row count.
 - Put the number on top of each bar (`plt.text(...)`).
- **Takeaway for Priya:** “We started with 150,000 businesses. We kept 5,000 open restaurants in Philadelphia. This is enough to train a model.”

Common mistakes in Class 1

Mistake	What goes wrong
Forget <code>lines=True</code> in <code>pd.read_json</code>	Code crashes or memory explodes.
Forget <code>.fillna('')</code> before <code>.str.contains</code>	Code crashes on rows with None in categories.
Forget <code>.copy()</code> after a filter	Pandas warns “SettingWithCopyWarning”.
Save to <code>/content/</code> instead of Drive	File disappears when Colab disconnects.
Pick a city with only 50 restaurants	Too small. Model cannot learn. Pick a top-5 city.

Self-check before Class 2

- ☐ business DataFrame loaded from JSON.
- ☐ Filter to “Restaurants” works.
- ☐ You picked ONE city and saved the name in a variable.
- ☐ At least 3 exploratory charts in your notebook.
- ☐ 1 polished chart for Priya.

- ☐ You saved `business_step1.parquet` to your Drive folder.
 - ☐ You wrote down (in markdown) what you did and WHY.
-

Class 2 — Encoding and Scaling

Scenario reminder: Priya looks at your filtered data. She is happy. But she says: “The model is a math model. It does not understand the word ‘Mexican’ or ‘\$\$’. Turn the words into numbers. Also, the `attributes` column is a mess — it is a nested JSON. Open it up.”

Your goal

Open the nested columns. Turn TEXT columns into numbers. Make all numeric columns about the same size.

Inputs

- `business_step1.parquet` from Class 1
- (The other JSON files are still in your folder. We will use review later.)

Outputs

- `business_step2.parquet` in Drive
 - 3+ exploratory charts
 - 1 explanatory chart for Priya
-

Phase A — Explore the data first

Exploratory chart 1 — Look INSIDE the `attributes` column

- **Question:** “What does an `attributes` cell look like? It is a dictionary, not a number.”
- **HINTS:**
 - Print one row: `print(business['attributes'].iloc[0])`.
 - You see a Python dictionary like `{'RestaurantsTakeOut': 'True', 'BusinessParking': '{\'garage\': False, \'street\': True, ...}', 'Alcohol': 'u\'full_bar\''}`.
 - Note the **quotes around True/False** — they are strings, not booleans. Strange!
- **What you learn:** This column is nested. We must expand it.

Exploratory chart 2 — How many attributes are missing?

- **Question:** “Many attributes are filled only for some restaurants. Which ones are useful?”
- **HINTS:**
 - First expand attributes (see Phase B Step 1).
 - Then `.isna().sum() / len(df) * 100` to get percent missing.
 - Plot as a bar chart, sorted.
- **What you learn:** Some attributes are 95% missing. We will drop them.

Exploratory chart 3 — `price_range` counts

- **Question:** “How many restaurants are cheap, medium, expensive?”
- **HINTS:**
 - Inside attributes, look at `RestaurantsPriceRange2`. It is 1, 2, 3, or 4 (number of \$ signs).
 - `value_counts().plot.bar()` on this column.

- **What you learn:** Most restaurants are

(medium).Veryfeware

\$\$.

Exploratory chart 4 — categories — top cuisines

- **Question:** “What kinds of food are in our city?”
- **HINTS:**
 - The categories column is a comma-separated string.
 - Split it: `df['categories'].str.split(', ').explode()`.
 - Then `value_counts().head(20).plot.bar()`.
- **What you learn:** Top cuisines: American, Pizza, Italian, Chinese, Mexican, etc.

Phase B — Encode and scale

Step 1 — Expand the nested attributes column

The attributes cell is a dictionary. We want each key to become its own column.

- **WHAT:** Turn the attributes column into many columns (one per attribute).
- **HINTS:**
 - Use `pd.json_normalize(df['attributes'])`.
 - If some rows have None in attributes, replace with empty dict first: `df['attributes'].apply(lambda x: x if isinstance(x, dict) else {})`.
 - The result is a NEW DataFrame. Concatenate it to df with `pd.concat([df, attrs], axis=1)`.
- **EXPECTED:** About 30 to 40 new columns: RestaurantsTakeOut, RestaurantsDelivery, Alcohol, WiFi, BusinessParking, etc.

Step 2 — Fix the weird string values

Yelp stores boolean values as **strings**, not Python booleans. You will see: - 'True', 'False', 'None' — these are strings! - "u'full_bar'" — a Python 2 unicode string literal. Strange. - '{"garage": False, "street": True}' — a string that LOOKS like a dictionary.

- **WHAT:** Clean up the values.
- **HINTS:**
 - For columns like RestaurantsTakeOut: replace 'True' → 1, 'False' → 0, anything else → NaN.
 - You can use `.map({'True': 1, 'False': 0})` or a small function.
 - For columns like Alcohol: the values are "u'full_bar'", "u'beer_and_wine'", "'none'". Use `.str.replace("u'", "").str.replace("'", "")` to clean.

Step 3 — Build the simple boolean flags

For Priya’s model, we want easy 0/1 flags. Pick these attributes:

New column	From which attribute
has_wifi	WiFi (1 if 'free' or 'paid', else 0)
accepts_credit_cards	BusinessAcceptsCreditCards (1 if True, else 0)
has_parking	BusinessParking (1 if any value inside is True)
outdoor_seating	OutdoorSeating
takes_reservations	RestaurantsReservations
good_for_groups	RestaurantsGoodForGroups
has_tv	HasTV

New column	From which attribute
noise_level_num	NoiseLevel → 1=quiet, 2=average, 3=loud, 4=very_loud
alcohol_num	Alcohol → 0=none, 1=beer_and_wine, 2=full_bar
price_range_num	RestaurantsPriceRange2 (already a number 1-4)

- **HINTS:**

- For has_parking, the value is a string like "{ 'garage': False, 'street': True }". Use `eval()` carefully or `ast.literal_eval()`. Or just check if "True" is in the string.
- Many of these will be missing. That is OK. Fill missing with 0 (or with the most common value) — your choice. Write it down.

Step 4 — Drop the messy original columns

Now that you have clean 0/1 flags, drop the old messy columns: `attributes`, `RestaurantsTakeOut` (original string version), `Alcohol`, etc. Keep only the clean ones.

- **HINTS:** `df.drop(columns=[...])`.

Step 5 — One-hot encode state

Even though we filtered to ONE city, the state column may have only 1 or 2 values. But the model still needs numbers.

- **WHAT:** Use one-hot encoding on state.
- **HINTS:**
 - `pd.get_dummies(df, columns=['state'], prefix='state')`.
- **NOTE:** If only one state remains after filtering, you may skip this. Discuss with your team.

Step 6 — Count categories per restaurant

- **WHAT:** Make a column `n_categories` = how many categories a restaurant has.
- **HINTS:**
 - `df['categories'].str.split(', ').str.len()`.
 - Be careful: some rows may have None. Handle them.
- **WHY:** A restaurant labeled “Mexican, Bars, Nightlife, Vegan, Gluten-Free” is different from one labeled just “Restaurants”. The count is a useful signal.

Step 7 — Log-transform review_count

The `review_count` column has a very long tail. Some restaurants have 5 reviews, some have 2,000.

- **WHAT:** Apply `np.log1p()`.
- **HINTS:** `df['log_review_count'] = np.log1p(df['review_count'])`.
- **WHY log1p and not log?** `log(0)` is `-inf`. `log1p(x) = log(x + 1)` is safe for zeros.

Step 8 — Scale numeric columns (preview only)

- **WHAT:** Use `StandardScaler` to make every numeric column have mean = 0, std = 1.
- **HINTS:**
 - `from sklearn.preprocessing import StandardScaler`.
 - `scaler = StandardScaler()`.
 - `scaler.fit_transform(df[numeric_cols])`.
- **WARNING:** This is for inspection only. In Class 5, scaling goes INSIDE the Pipeline. Do NOT save the scaled version as your final file.

Step 9 — Save

- **WHAT:** Save the UNSCALED version. `df.to_parquet('/content/drive/MyDrive/yelp_lab/business_step2.pa`
-

Phase C — Make ONE chart for Priya

Priya's chart — “Cuisines in our city”

A horizontal bar chart of the top 15 cuisine types in your city.

- **HINTS:**
 - `df['categories'].str.split(', ').explode().value_counts().`
 - **Remove** the word “Restaurants” itself (it is on every row, not useful).
 - `.head(15).plot.barh().`
 - **Title:** “Top 15 cuisines in Philadelphia - American and Pizza dominate”.
 - **X-label:** Number of restaurants.
 - **Y-label:** Cuisine.
 - **Takeaway:** “Most of our market is American, Pizza, and Italian. Asian and Mexican are smaller but growing categories.”
-

Common mistakes in Class 2

Mistake	What goes wrong
Treat 'True' (string) as True (boolean)	Wrong values. The flag is always 0.
Forget to replace None in attributes	Crash on <code>pd.json_normalize</code> .
Scale BEFORE train/test split	Leakage. Scaler sees test data.
One-hot encode categories directly	1,200+ columns. Table explodes.
Forget <code>np.log1p</code> and use <code>np.log</code> on zeros	<code>np.log(0) = -inf</code> . Crash.
Save the scaled version	Class 5 scales again inside the pipeline. Double scaling = wrong numbers.

Self-check before Class 3

- ☐ `attributes` column is expanded into many columns.
 - ☐ Boolean flags (`has_wifi`, `has_parking`, etc.) are clean 0/1.
 - ☐ `n_categories` exists.
 - ☐ `log_review_count` exists.
 - ☐ At least 3 exploratory charts.
 - ☐ 1 chart for Priya.
 - ☐ `business_step2.parquet` saved to Drive.
-

Class 3 — Feature Engineering

Scenario reminder: Priya says: “The raw columns are not enough. The TRULY useful columns are not there. We must MAKE them. For example, **how many hours per week is the restaurant open?** A 24-hour diner is very different from a brunch place. Build features like this.”

Your goal

Make NEW columns from the existing ones. These will help the model predict the star rating.

Inputs

- business_step2.parquet (with expanded attributes)
- yelp_academic_dataset_review.json (the review file — we will sample it)
- The hours column inside business_step1.parquet (nested JSON)

Outputs

- business_step3.parquet in Drive
 - 3+ exploratory charts
 - 1 explanatory chart for Priya
-

Phase A — Explore the data first

Exploratory chart 1 — hours column structure

- **Question:** “What does a hours cell look like?”
- **HINTS:**
 - Print one row: `print(business['hours'].iloc[0]).`
 - You see something like `{ 'Monday': '11:0–22:0', 'Tuesday': '11:0–22:0', ..., 'Sunday': '11:0–21:0' }`.
- **What you learn:** Hours are stored as a nested dictionary. We must parse the strings to get total hours per week.

Exploratory chart 2 — stars vs review_count

- **Question:** “Do popular restaurants (many reviews) have higher or lower stars?”
- **HINTS:**
 - Scatter plot: `plt.scatter(df['review_count'], df['stars']).`
 - Use `alpha=0.3` to see the density.
- **What you learn:** There may be a small positive correlation. But many cheap places with few reviews also have 4.5 stars.

Exploratory chart 3 — Latitude / Longitude map of your city

- **Question:** “Where in the city are the restaurants?”
- **HINTS:**
 - `plt.scatter(df['longitude'], df['latitude'], c=df['stars'], cmap='RdYlGn', alpha=0.5).`
 - Add a colorbar.
- **What you learn:** Are downtown restaurants higher-rated? Are suburb ones lower? Geographic patterns.

Exploratory chart 4 — Hours open per week (after you compute it in Step 1)

- **HINTS:** Histogram, `bins=30`.
 - **What you learn:** Most restaurants open 40-80 hours per week. Some are 24/7. Some are weekend-only.
-

Phase B — Engineer the features

Step 1 — Parse the hours column

This is the **hardest** feature. The cell value is a dictionary. Each value is a string like "11:0–22:0".

Sub-step 1a — Handle missing values. - **HINTS:** Some rows have None in hours. Replace with an empty dict, or skip them.

Sub-step 1b — Write a function to compute hours per day.

```
def hours_for_one_day(time_string):
    # time_string looks like "11:0-22:0" (open 11am to 10pm)
    if time_string is None or time_string == '':
        return 0
    open_str, close_str = time_string.split('-')
    open_h, open_m = open_str.split(':')
    close_h, close_m = close_str.split(':')
    open_time = int(____) + int(____) / 60    # YOU fill in
    close_time = int(____) + int(____) / 60
    if close_time < open_time:
        close_time += 24    # closes after midnight
    return close_time - open_time
```

• **HINTS:**

- Test on one value first: `print(hours_for_one_day('11:0-22:0'))` should be 11.0.
- Test on '17:0-1:0' (open 5pm, closes 1am) should be 8.0.

Sub-step 1c — Sum over the 7 days.

```
def hours_per_week(hours_dict):
    if not isinstance(hours_dict, dict):
        return 0
    days = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
    total = 0
    for day in days:
        if day in hours_dict:
            total += hours_for_one_day(____)    # YOU fill in
    return total
```

• **HINTS:**

- Apply this to your hours column: `df['hours_open_per_week'] = df['hours'].apply(hours_per_week)`.

- **EXPECTED:** Values 0 to ~168 (168 = 24/7). Most are 50 to 100.

Step 2 — Day-of-week features

Make these new columns:

New column	What it is
open_on_monday	1 if Monday is in the hours dict, else 0
open_on_sunday	1 if Sunday is in the hours dict, else 0
n_days_open	Number of days per week the restaurant is open
opens_late	1 if any day closes after midnight
opens_early	1 if any day opens before 8am

• **HINTS:**

- Use `.apply()` with a small function.
- For `opens_late`, look at the close time of each day. If `close < open`, it crosses midnight.

- **WHY:** A breakfast place (opens early) is rated differently from a late-night bar. These features capture that.

Step 3 — Geographic features

New column	What it is
dist_from_center	Distance in km from the city center (lat/lon)
lat_rounded	Latitude rounded to 2 decimals (creates “zones”)
lon_rounded	Longitude rounded to 2 decimals

• **HINTS:**

- For city center: find the mean lat/lon of all restaurants. Or look up the real city center.
- For distance, use the Haversine formula (you may have learned it in the Olist scenario):

```
import numpy as np
def haversine(lat1, lon1, lat2, lon2):
    R = 6371 # Earth radius in km
    phi1 = np.radians(____) # YOU fill in
    phi2 = np.radians(____)
    dphi = np.radians(____ - ____)
    dlam = np.radians(____ - ____)
    a = np.sin(dphi/2)**2 + np.cos(phi1)*np.cos(phi2) * np.sin(dlam/2)**2
    return 2 * R * np.arcsin(np.sqrt(a))
```

- **WHY:** Restaurants near the city center may be rated differently than ones far away. The model can learn the pattern.

Step 4 — Category-based features

New column	What it is
is_pizza	1 if “Pizza” in categories
is_chinese	1 if “Chinese” in categories
is_italian	1 if “Italian” in categories
is_mexican	1 if “Mexican” in categories
is_american	1 if “American (Traditional)” or “American (New)” in categories
is_bar	1 if “Bars” or “Pubs” in categories
is_fast_food	1 if “Fast Food” in categories
is_cafe	1 if “Cafes” or “Coffee & Tea” in categories
is_vegan	1 if “Vegan” or “Vegetarian” in categories

• **HINTS:**

- Use `df['categories'].str.contains('Pizza', case=False, na=False).astype(int)`.

- **WHY:** The model needs to know the cuisine type. Pizza places have different rating patterns than vegan places.

Step 5 — Review-based features (the IMPORTANT ones)

Now we use the review file. This file is BIG (5 GB). We must sample it.

Sub-step 5a — Load reviews for restaurants in your city only.

This is tricky. The review file has 6 million rows. We want only reviews of OUR restaurants (the ~5,000 we kept).

• **HINTS:**

- Get the set of business IDs we kept: `kept_ids = set(df['business_id'])`.
- Read the JSON file in **chunks**: `pd.read_json(..., lines=True, chunksize=100000)`.
- For each chunk, filter to rows where `business_id` is in `kept_ids`. Append to a list. Concat at the end.
- Skeleton:

```
chunks = []
reader = pd.read_json('yelp_academic_dataset_review.json', lines=True, chunksize=____)
for chunk in reader:
```



```

keep = chunk[chunk['business_id'].isin(__)]
chunks.append(keep)
reviews = pd.concat(chunks, ignore_index=True)

```

- **EXPECTED:** A few hundred thousand reviews for our restaurants. Manageable.

Sub-step 5b — Compute review-based features per restaurant.

New column	What it is
mean_review_length	Average length (in characters) of the reviews of this restaurant
mean_review_sentiment_score	Simple sentiment: count of positive words minus negative words
review_velocity	Number of reviews per year (review_count / years_active)
first_review_year	Year of the first review
last_review_year	Year of the last review

- **HINTS:**
 - Group reviews by business_id: `reviews.groupby('business_id')`.
 - For mean_review_length: `reviews['text'].str.len()` then group and average.
 - For mean_review_sentiment_score: define a small word list. Example: positive = {'good', 'great', 'amazing', 'love', 'delicious'}. Negative = {'bad', 'awful', 'cold', 'slow', 'rude'}. Count occurrences. (We will improve this in Module 7.)
 - For review_velocity: `review_count / (last_review_year - first_review_year + 1)`.
- **WARNING — Leakage!**
 - mean_review_sentiment_score uses review TEXT. In production, we predict for NEW restaurants with NO reviews. So this feature is NOT available at prediction time!
 - **For Module 3 we will compute it anyway**, but mark it clearly. In Module 4 we will split features into two sets: “available at launch” and “available after some reviews”. The model will use only the launch-time features for the cold-start problem.

Step 6 — Save

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/yelp_lab/business_step3.parquet')`.

Phase C — Make ONE chart for Priya

Priya’s chart — “Hours open per week vs star rating”

A box plot showing the star rating distribution for restaurants in different “hours per week” buckets.

- **HINTS:**
 - Bucket hours per week: `pd.cut(df['hours_open_per_week'], bins=[0, 30, 60, 90, 120, 168])`.
 - Make a box plot: `sns.boxplot(x=buckets, y=df['stars'])`.
- **Title:** “Star rating by weekly opening hours — restaurants open 60-90 h/week have the highest median rating”.
- **Takeaway for Priya:** “Restaurants that are open ‘normal’ hours (60-90 per week) score best. 24/7 places (often fast food) score lower. Closed-most-of-the-week places score highest only if they are specialty (brunch, weekend pop-ups).”

Common mistakes in Class 3

Mistake	What goes wrong
Forget to handle midnight-crossing in <code>hours_for_one_day</code>	Negative hours. Or 0.
Read the full review file without chunking	Colab runs out of memory.
Forget to filter reviews to kept <code>business_ids</code> before concat	Concat the whole 5 GB file. Crash.
Use review text features as model input in M4	Leakage — these are NOT available for new restaurants.
Use Euclidean distance instead of Haversine	Wrong distances in km.

Self-check before Class 4

- ☐ `hours_open_per_week` exists. Values 0 to 168.
- ☐ At least 5 cuisine flags exist (`is_pizza`, etc.).
- ☐ Review-based features exist (`mean_review_length`, etc.).
- ☐ Geographic feature (`dist_from_center`) exists.
- ☐ 3+ exploratory charts.
- ☐ 1 explanatory chart.
- ☐ `business_step3.parquet` saved to Drive.

Class 4 — Feature Selection

Scenario reminder: Now you have ~40 columns. Priya says: “Too many. Some are duplicates. Some are useless. I want 15-20 GOOD columns. Pick them.”

Your goal

Pick the best 15-20 columns. Drop the rest. Justify every choice.

Inputs

- `business_step3.parquet`

Outputs

- `business_step4.parquet` in Drive (only the selected columns + stars)
- 3+ exploratory charts (correlation heatmap, mutual info bars, importance bars)
- A short markdown report explaining your choices

Phase A — Explore the data first

Exploratory chart 1 — Correlation heatmap of numeric columns

- **Question:** “Which columns say the same thing as each other?”
- **HINTS:**
 - Compute `df[numeric_cols].corr()`.
 - Plot with `sns.heatmap()`.
 - Use `annot=True` and `fmt='.2f'`.
- **What you learn:** Pairs with `lcorr` > 0.9 are redundant. For example, `review_count` and `log_review_count` are perfectly correlated — keep only one.

Exploratory chart 2 — Mutual information bar chart

- After you compute mutual info (Step 4 below), plot it.
- **HINTS:**
 - Sort the values, then bar chart.
- **What you learn:** Which columns predict stars strongest.

Exploratory chart 3 — Random Forest feature importance

- After you train the RF (Step 5 below), plot the importances.
 - **HINTS:** `pd.Series(rf.feature_importances_, index=...).sort_values().plot.barh()`.
 - **What you learn:** A second opinion on feature importance.
-

Phase B — Select features

Step 1 — Split into train and test FIRST

- **WHAT:** Use `train_test_split`.
- **HINTS:**
 - `from sklearn.model_selection import train_test_split.`
 - `X = df.drop('stars', axis=1); y = df['stars'].`
 - Pass: `test_size=0.2, random_state=42.`
 - For regression (continuous target) you do NOT use `stratify=y` directly. If you treat it as multi-class (9 levels), you can stratify.
- **WHY:** All next steps use TRAIN ONLY. Otherwise leakage.

Step 2 — Remove low-variance columns

- **WHAT:** Drop columns where almost all values are the same.
- **HINTS:**
 - `from sklearn.feature_selection import VarianceThreshold.`
 - `VarianceThreshold(threshold=0.01)` then `.fit_transform(X_train[numeric_cols]).`
- **EXAMPLE:** If 99% of restaurants have `state == 'PA'` after city filtering, this column has no variance. Drop it.

Step 3 — Remove highly correlated columns

- **WHAT:** Drop one of each pair where `lccorr > 0.9`.
- **HINTS:**
 - Compute correlation matrix.
 - Get the upper triangle with `np.triu(...)`.
 - For each column, if any of its upper-triangle correlations is `> 0.9`, mark it for dropping.
- **EXAMPLE:** `review_count` and `log_review_count` are perfectly correlated. Keep `log_review_count` (better shape).

Step 4 — Rank by mutual information

- **WHAT:** Score each column by how much it tells you about stars.
- **HINTS:**
 - For regression: `from sklearn.feature_selection import mutual_info_regression.`
 - `mi = mutual_info_regression(X_train_numeric, y_train).`
 - Put in a Series, sort.
- **EXPECTED:** `mean_review_length`, `price_range_num`, `log_review_count`, `hours_open_per_week` should be near the top.

Step 5 — Random Forest importance (second opinion)

- **WHAT:** Train a small RF, look at `feature_importances_`.
- **HINTS:**
 - `from sklearn.ensemble import RandomForestRegressor` (for regression).
 - `RandomForestRegressor(n_estimators=50, max_depth=8, n_jobs=-1, random_state=42)`.
 - `.fit(X_train, y_train)`.
 - Look at `.feature_importances_`.

Step 6 — Pick the final 15-20 columns

- **WHAT:** Combine the rankings. Pick columns that:
 - Are high in mutual info, AND
 - Are high in RF importance, AND
 - Were not dropped by variance/correlation pruning.
- **WRITE DOWN:** In your notebook, list the columns. Explain in 1 sentence why each one is in.

Step 7 — Mark the leakage features

For each kept feature, label it: - **AT-LAUNCH:** Available for a brand-new restaurant (category, price, location, hours). - **AFTER-REVIEWS:** Only available after some reviews come in (mean_review_length, mean_review_sentiment_score, review_velocity).

In Module 4, we will train TWO models: one for cold-start (AT-LAUNCH only), one for warm-start.

Step 8 — Save

- **HINTS:** Keep only the selected columns + stars. Save as `business_step4.parquet` to Drive.
-

Phase C — Make ONE chart for Priya

Priya's chart — "These are the 15 most important columns"

A horizontal bar chart of your top 15 features and their importance score. **Color the bars:** green for "AT-LAUNCH" features, orange for "AFTER-REVIEWS" features.

- **HINTS:**
 - Use the RF importance.
 - `sort_values()` then `.head(15)` then `.plot.barh()`.
 - Set the color list based on each feature's label.
 - **Title:** "Top 15 predictors of star rating - green = available at launch".
 - **Y-axis:** column names.
 - **X-axis:** Random Forest importance.
 - **Takeaway:** "The strongest signals are review-based (orange). But for new restaurants we only have the green ones. We must work hard to make them good."
-

Common mistakes in Class 4

Mistake	What goes wrong
Compute mutual info on the FULL data	Leakage.
Use <code>mutual_info_classif</code> instead of <code>mutual_info_regression</code>	Wrong scoring for our continuous target.
Drop columns without writing why	Module 4 students will not understand.

Mistake	What goes wrong
Keep too many columns (50+)	Slow training. Overfitting risk.
Forget to mark leakage features	Module 4 model “works” but fails in production.

Self-check before Class 5

- ☐ Train/test split done FIRST.
- ☐ 15-20 columns remain + stars.
- ☐ Each kept column is labeled AT-LAUNCH or AFTER-REVIEWS.
- ☐ 3+ exploratory charts.
- ☐ 1 chart for Priya.
- ☐ business_step4.parquet saved to Drive.

Class 5 — Pipelines

Scenario reminder: Priya says: “Your cleaning code is in 4 different notebooks. When a new restaurant signs up tomorrow, you cannot run 4 notebooks. We need ONE object that does everything — from raw JSON row to a star prediction.”

Your goal

Put EVERY cleaning step inside ONE Pipeline object. Production-ready code.

Inputs

- business_step4.parquet (selected columns)

Outputs

- yelp_pipeline.joblib saved in Drive
- 1 scatter plot of predicted vs actual stars
- 1 residual plot
- A pipeline that takes RAW input and produces predictions

Phase A — Explore the data first

Exploratory chart 1 — Predicted vs actual scatter (after training)

- After Step 6, predict on the test set.
- **HINTS:**
 - `y_pred = pipeline.predict(X_test).`
 - `plt.scatter(y_test, y_pred, alpha=0.3).`
 - Add a diagonal line `y = x` for reference.
- **What you learn:** Are predictions close to the diagonal? Or scattered?

Exploratory chart 2 — Residual plot

- **HINTS:**
 - `residuals = y_test - y_pred.`
 - `plt.scatter(y_pred, residuals).`

- Add a horizontal line at 0.
 - **What you learn:** Are residuals random (good)? Or do they have a pattern (bad — model misses something)?
-

Phase B — Build the pipeline

Step 1 — Decide which columns are numeric and which are categorical

- **HINTS:** Make two lists.
- **EXAMPLE:**
 - numeric: price_range_num, n_categories, log_review_count, hours_open_per_week, dist_from_center, latitude, longitude, mean_review_length, review_velocity, n_days_open.
 - categorical: state (if you kept it). Or none, if all flags are 0/1 already.
 - The 0/1 flags (like has_wifi, is_pizza) are technically numeric. Treat them as numeric.

Step 2 — Build the numeric mini-pipeline

- **WHAT:** 2 steps: imputer (fill missing) + scaler.
- **HINTS:**
 - from sklearn.pipeline import Pipeline.
 - from sklearn.impute import SimpleImputer.
 - from sklearn.preprocessing import StandardScaler.
 - Skeleton:

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('scaler', StandardScaler()),
])
```
 - Fill in the blank: best strategy for numeric? ('median')

Step 3 — Build the categorical mini-pipeline (if you have any categorical)

- **WHAT:** 2 steps: imputer + one-hot encoder.
- **HINTS:**
 - from sklearn.preprocessing import OneHotEncoder.
 - Skeleton:

```
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('onehot', OneHotEncoder(handle_unknown='___', sparse_output=___)),
])
```
 - Fill in blanks. (Strategy 'most_frequent', handle_unknown 'ignore', sparse_output False.)
- **WHY handle_unknown='ignore'?** In production, a new restaurant in a new state may appear. We do not want the code to crash.

Step 4 — Combine into a ColumnTransformer

- **HINTS:**
 - from sklearn.compose import ColumnTransformer.
 - It takes a list of tuples: (name, transformer, list_of_columns).

Step 5 — Add the model on top

For a REGRESSION problem (predict a number 1.0 to 5.0):

- **HINTS:**
 - from sklearn.linear_model import Ridge.
 - Ridge(alpha=1.0, random_state=42).

- Put it in a Pipeline with the preprocessor.

If you prefer multi-class (9 levels: 1.0, 1.5, 2.0, ..., 5.0): - from sklearn.ensemble import RandomForestClassifier.

Recommendation for Module 3: Use Ridge regression. It is simple. In Module 4 we will try better models (Random Forest Regressor, Gradient Boosting).

Step 6 — Train and evaluate

- **HINTS:**
 - full_pipeline.fit(X_train, y_train).
 - y_pred = full_pipeline.predict(X_test).
 - from sklearn.metrics import mean_absolute_error, r2_score.
 - Print MAE (Mean Absolute Error): “on average, we are off by X stars”.
 - Print R² score.
- **EXPECTED:** MAE around 0.5 to 0.7 stars. R² around 0.2 to 0.4. (Module 4 will improve this.)

Step 7 — Save the trained pipeline

- **HINTS:**
 - import joblib.
 - joblib.dump(full_pipeline, '/content/drive/MyDrive/yelp_lab/yelp_pipeline.joblib').

Phase C — Make ONE chart for Priya

Priya’s chart — “How close are our predictions?”

A scatter plot of y_test (actual stars) vs y_pred (predicted stars).

- **HINTS:**
 - plt.scatter(y_test, y_pred, alpha=0.3).
 - Add a red diagonal line: plt.plot([1, 5], [1, 5], 'r--').
 - X-label: “Actual stars”.
 - Y-label: “Predicted stars”.
- **Title:** “Predicted vs actual stars - MAE = 0.6 (we are off by about half a star on average)”.
- **Takeaway for Priya:** “Half a star is not perfect, but it is far better than ‘unknown’. We can already use this for cold-start ranking. Module 4 will improve the accuracy.”

Common mistakes in Class 5

Mistake	What goes wrong
Build the pipeline AFTER you scaled the data manually	Pipeline scales it twice. Numbers wrong.
Forget sparse_output=False in OneHotEncoder	Some downstream code breaks.
Use LogisticRegression for regression target	Logistic is for 0/1 classification. Wrong tool.
Forget random_state	Results change every run. Hard to debug.
Use mean_review_sentiment_score in the cold-start model	Leakage — not available for a new restaurant.

Self-check before Class 6

- ☐ Pipeline has preprocessor + model.
- ☐ Preprocessor has numeric + (maybe) categorical transformers.

- ☐ `handle_unknown='ignore'` set (if categorical pipeline used).
- ☐ Predicted-vs-actual chart in notebook.
- ☐ Residual chart in notebook.
- ☐ `yelp_pipeline.joblib` saved to Drive.

Class 6 — End-to-End Lab (THE BIG ONE)

Scenario reminder: Priya is in the meeting room. She wants the FINAL clean dataset on her desk in 90 minutes. This is the lab.

Your goal

Take 5 raw JSON files. Produce ONE final `.parquet` file with the exact 22-column schema. Plus a 1-page findings report.

Time

90 minutes of focused work.

Inputs

- The 5 raw JSON files in your Drive folder

Outputs

- `yelp_restaurants_clean.parquet` (~5,000 to ~10,000 rows × 22 columns) in Drive
- `findings.md` (~1 page)
- Your full Colab notebook

Phase A — Explore the data first (10 minutes)

You will reuse charts from Classes 1-5. Pick 3 of them. Put them all on ONE PAGE of your notebook with markdown headers: 1. stars distribution (the target) 2. Top 15 cuisines in your city 3. Top 15 most important features (with AT-LAUNCH vs AFTER-REVIEWS colors)

These 3 charts tell Priya the whole story.

Phase B — Build the final dataset (70 minutes)

Required output schema

Your `yelp_restaurants_clean.parquet` MUST have these 22 columns, with these names and types:

#	Column	Type	Source
1	<code>business_id</code>	string	business
2	<code>state</code>	string	business
3	<code>city</code>	string	business
4	<code>latitude</code>	float	business
5	<code>longitude</code>	float	business
6	<code>dist_from_center</code>	float	engineered (Haversine)

#	Column	Type	Source
7	n_categories	int	engineered (split categories)
8	price_range_num	int (1-4)	from at-tributes.RestaurantsPriceRange
9	has_wifi	int (0/1)	from attributes.WiFi
10	has_parking	int (0/1)	from at-tributes.BusinessParking
11	accepts_credit_cards	int (0/1)	from at-tributes.BusinessAcceptsCredit
12	outdoor_seating	int (0/1)	from at-tributes.OutdoorSeating
13	alcohol_num	int (0-2)	from attributes.Alcohol
14	noise_level_num	int (1-4)	from at-tributes.NoiseLevel
15	hours_open_per_week	float	engineered from hours
16	n_days_open	int (0-7)	engineered from hours
17	review_count	int	business
18	log_review_count	float	engineered
19	review_velocity	float	engineered (reviews per year)
20	mean_review_length	float	engineered (from review.json)
21	mean_review_sentiment_score	float	engineered (from review.json — for Module 7!)
22	stars	float (1.0-5.0)	TARGET (from business)

Lab phases (timed)

Phase	Time	What you do
1. Load + filter	15 min	Load business JSON. Filter to Restaurants, one city, is_open.
2. Expand attributes	10 min	pd.json_normalize. Build 0/1 flags. Build price_range_num.
3. Parse hours	10 min	Compute hours_open_per_week, n_days_open.
4. Geo features	5 min	Compute dist_from_center (Haversine).
5. Category counts	5 min	Compute n_categories.
6. Read reviews (chunked)	15 min	Filter review.json to kept business_ids. Compute mean length, sentiment, velocity.
7. Validate schema + save	5 min	Check 22 columns. Save .parquet.
8. Findings	10 min	Write findings.md.

Total: 90 minutes.

Phase C — Findings report for Priya (10 minutes)

Write findings.md with these sections:

Final numbers

- City: ____ (e.g., Philadelphia)
- Final row count: ____
- Mean stars: ____ (should be around 3.5)
- Number of AT-LAUNCH features: ____
- Number of AFTER-REVIEWS features: ____

Top 3 insights

1. ____ (e.g., "Pizza restaurants score 0.4 stars below the city average.")
2. ____ (e.g., "Restaurants open 60-90 hours/week score highest.")
3. ____ (e.g., "Distance from center is weakly negatively correlated with stars.")

One question to investigate in Module 4

- ____ (e.g., "Can we beat MAE = 0.6 with Gradient Boosting?")

One chart that summarizes everything

Embed your most important chart (the top 15 features one, with AT-LAUNCH vs AFTER-REVIEWS colors).

Grading rubric (100 points)

Item	Points
All 22 columns exist with the right names and dtypes	25
Cleaning decisions written in markdown	10
Pipeline is reproducible (re-run from 5 JSON files in ONE command)	15
hours_open_per_week correctly handles midnight crossing	10
Review file read with chunking (no memory crash)	10
At least 3 exploratory + 1 explanatory chart per class	15
findings.md has insights, not just numbers	10
AT-LAUNCH vs AFTER-REVIEWS labels in your notes	5

Submit

Upload to module-3/class_6/submissions/<YourTeamName>/: - yelp_restaurants_clean.parquet - Your Colab notebook (File --> Download --> Download .ipynb) - findings.md

Tips for the whole module

1. **Do NOT copy-paste code from this guide.** This guide gives you HINTS only. Write the code yourself. You will learn much faster.
2. **Save your notebook to Drive often.** Colab disconnects after 12 hours.
3. **Save intermediate .parquet files to Drive** (business_step1, business_step2, etc.). If something breaks, you do not redo everything.
4. **Pair-program.** One student types, one reads. Switch every 20 minutes.
5. **Make a chart BEFORE you write cleaning code.** You must SEE the problem before you can fix it.

6. **Make a chart AFTER each step.** Confirm your code did what you expected.
7. **Always pass `lines=True` to `pd.read_json`** for Yelp data. The file is JSON Lines, not standard JSON.
8. **Filter as early as possible.** Restaurants + one city + open = the only data you need. Doing the filter on row 1 saves memory and time.
9. **For the review file, always use `chunksize`.** Never load 5 GB at once.
10. **Mark leakage features.** Any feature built from review text is NOT available for a new restaurant.
11. **Ask the mentor early.** If you are stuck for 20 minutes, ask. Do not waste an afternoon.

Good luck.